Start here: HTTPie CLI

HTTPie is a command-line HTTP client for talking to APIs: http and https send a request built from short key=value arguments and print a colored, formatted response. This repo (version 3.2.4) is the Python package behind those commands and the httpie management tool.

```
git clone https://github.com/httpie/cli.git && cd cli
python3 -m venv venv && ./venv/bin/pip install -e '.[dev]'
./venv/bin/http --offline pie.dev/post name=Ada age:=36
```

--offline builds and prints the request without sending it, which makes it the quickest way to see what your change does:

```
POST /post HTTP/1.1
Content-Type: application/json
User-Agent: HTTPie/3.2.4
...
{"name": "Ada", "age": 36}
```

Users install it with pip install httpie (Python 3.7+). Full user docs live in docs/README.md and at https://httpie.io/docs/cli.

Repo map

Path	What lives there
httpie/__main__.py, httpie/core.py	Entry point, error handling, the main request loop
httpie/cli/	Argument definition (definition.py), parsing (argparser.py), request items (requestitems.py), nested JSON syntax (nested_json/)
httpie/client.py	Turns parsed args into a requests request, sends it, follows redirects
httpie/output/	Formatting, colors, streaming to the terminal
httpie/sessions.py, config.py	Saved sessions and the config directory
httpie/plugins/	Plugin base classes and loader
httpie/manager/	The httpie cli ... command (plugins, sessions, update checks)
tests/	pytest suite; tests/utils/__init__.py has the http() test helper

Day 1: run the tests

The Makefile wraps the setup. make all creates ./venv, installs .[dev] and .[test] plus an editable

HTTPie, and runs the tests. After that:

```
make test               # pytest over httpie/ and tests/ (doctests included)
make codestyle          # flake8, required to pass in CI
python -m pytest tests/test_uploads.py::TestMultipartFormDataFileUpload::test_upload_ok
```

On Windows, CI skips make: python -m pip install -e .[dev], then python -m pytest ./httpie ./tests.

How a request flows

1. httpie/__main__.py:main calls httpie.core.main (core.py:146), which hands the parser from cli/definition.py:956 to raw_main (core.py:32).
2. raw_main loads plugins, prepends default_options from the config file, then parses args with HTTPieArgumentParser (cli/argparser.py:138).
3. program (core.py:170) iterates over collect_messages (client.py:43) and yields the prepared request.

To add an option, define it in cli/definition.py, handle it in argparser.py or client.py, and test it with the http() helper, using --offline when no response is needed.